

## Лабораторна робота №5: Комплексний аналіз складності та оцінка якості програмного забезпечення за допомогою кількісних метрик

*Мета роботи:* Опанування практичних навичок кількісної оцінки складності програмного забезпечення та прогнозування трудовитрат на розробку шляхом застосування метрик Холстеда, Мак-Кейба та Чепіна; навчитися інтерпретувати отримані технічні показники для прийняття управлінських рішень щодо якості коду, необхідності рефакторингу та планування ресурсів на тестування.

### **Програмна частина завдання:** Розробка автоматизованої системи метричного аудиту ПЗ

#### *Контекст (Scenario):*

Компанія «ElectroSoftware» стикається з великим обсягом Legacy-коду. Керівництво прийняло рішення відмовитися від ручних розрахунків. Вам, як команді розробників внутрішніх інструментів, потрібно створити програмний продукт (Desktop, Web або CLI застосунок), який автоматизує аудит коду за трьома ключовими методиками.

#### *Вимоги до програмного продукту:*

- 1) Модуль лексичного аналізу (За Холстедом):
  - Програма повинна приймати на вхід файл із кодом (наприклад, .py, .js або .cpp).
  - Автоматично ідентифікувати та підраховувати унікальні оператори (n1, N1) та операнди (n2, N2).
  - Виводити розраховані значення Об'єму (V), Складності (D), Зусиль (E) та прогнозовану кількість помилок (B).
- 2) Модуль структурного аналізу (За Мак-Кейбом):
  - Програма має аналізувати логічну структуру функцій.
  - Мінімальна вимога: Автоматичний розрахунок цикломатичного числа  $V(G)$  шляхом підрахунку предикатних вузлів.
  - Додаткова перевага (High Level): Візуалізація графу потоку управління (CFG) у графічному вигляді.
- 3) Модуль інформаційного аналізу (За Чепіним):
  - Інтерфейс для класифікації змінних (користувач позначає тип змінної: P, M, C або T).
  - Автоматичний розрахунок інтегрального показника Q згідно з ваговими коефіцієнтами.
- 4) Модуль управлінської звітності (Decision Support):
  - На основі розрахованих даних система повинна автоматично генерувати текстову пораду: «Release Recommended» (якщо  $V(G) \leq 10$  та  $B < 0.5$ ) або «Refactoring Required» (якщо показники перевищують норму).

#### *Очікуваний результат:*

- Працюючий програмний код та коротка демонстрація його роботи на прикладі будь-якого довільного алгоритму.

Порада: Можна використовувати готові бібліотеки для парсингу, щоб зосередитись на реалізації самих алгоритмів метрик, а не на написанні компілятора з нуля. Приклади:

- Python: ast, inspect, lizard.
- Java: JavaParser, Checkstyle API.
- C#: Roslyn API (Microsoft.CodeAnalysis).
- JS: Esprima, Acorn.

### *Вимоги до графічного інтерфейсу (GUI Module)*

Програма повинна мати інтуїтивно зрозумілу візуальну оболонку, яка дозволяє користувачу (Project Manager або QA Lead) ефективно проводити аудит без редагування коду аналізатора.

Ключові компоненти інтерфейсу:

#### 1) Dashboard (Головна панель):

- Вікно для завантаження або вставки (Copy-Paste) вихідного коду програми, що досліджується.
- Кнопка «Запустити комплексний аудит».
- Панель вибору мови програмування (якщо підтримується декілька).

#### 2) Visual Metrics Explorer (Візуалізація метрик):

- Табличне представлення: Відображення списку знайдених операторів та операндів (для Холстеда) з можливістю сортування за кількістю входжень.
- Інтерактивна таблиця Чепіна: Список усіх виявлених змінних у коді, де користувач може через випадаючий список (Dropdown) обрати категорію (Р, М, С, Т), після чого показник Q перераховується миттєво.
- Графічне представлення: Виведення результатів у вигляді діаграм (наприклад, порівняння  $V(G)$  вашого модуля із середнім допустимим значенням за стандартом NIST).

#### 3) Graph Visualization (Опційно, для високої оцінки):

- Вікно з візуалізованим Графом потоку управління (CFG), де предикатні вузли (точки прийняття рішень) виділені окремим кольором.

#### 4) Reporting Hub (Центр звітів):

- Окрема зона з висновком «Managerial Report».
- Кольорова індикація ризику (наприклад: зелений — «Safe for Release», жовтий — «Warning», червоний — «Critical: Refactoring Required»).
- Можливість експортувати результати аналізу в PDF або JSON файл.

### *Технологічний стек (рекомендований для студентів):*

- Python: Tkinter, PyQt або Streamlit (для швидкого Web-GUI).
- JavaScript: React або Vue.js (якщо це Web-застосунок).
- C#: WPF або WinForms.

### *Технічні вимоги до обраного коду:*

- Для метрики Холстеда: Наявність арифметичних операцій, логічних порівнянь та роботи з вхідними параметрами (мінімум 10-15 унікальних токенів).
- Для метрики Мак-Кейба: Мінімум 3-5 точок прийняття рішень (if, elif, else або switch/case), включаючи вкладені конструкції, для побудови нетривіального графу керування (CFG).
- Для метрики Чепіна: Використання змінних різних категорій: вхідні (Р), керуючі (С), модифіковані (М) та несуттєві/службові (Т).

## ПРАКТИЧНА ЧАСТИНА

### Завдання 1. Аналіз складності та оцінка інтелектуальних зусиль розробки ПЗ за методом Холстеда

*Контекст (Кейс):* Ви, як Project Manager у компанії «ElectroSoftware», отримали замовлення на розробку деякого модуля для існуючого продукту замовника. Представники замовника надали вам прототип коду (обрати самостійно на будь-якій мові розробки) для якого вам, як РМ-у проєкту, потрібно обґрунтувати замовнику:

- Скільки «чистого часу» (Effort) потрібно на написання такого коду.
- Яка ймовірність виникнення помилок (багів) у цьому модулі.
- Чи не є код занадто складним для підтримки (Maintainability).

#### Умова:

- Лексичний аналіз: Розбити програмний код обраного варіанту на оператори та операнди. Сформувати таблиці унікальних елементів.
- Розрахунок базових метрик: Обчислити словник (n), довжину (N) та об'єм (V) програми.
- Оцінка складності: Розрахунок показника складності (D) та інтелектуальних зусиль (E).
- Прогнозний аналіз:
  - Розрахувати теоретичний час розробки (T) в годинах.
  - Оцінити кількість потенційних помилок (B) за формулою Холстеда.

$$B = \frac{E^{2/3}}{3000}$$

- Managerial Report (Висновок): Написати короткий звіт для замовника: чи рекомендуєте ви цей код до релізу, чи він потребує рефакторингу через надмірну складність.

#### *Контрольні запитання для захисту*

- Економічний аспект: Як збільшення кількості унікальних операндів (n2) впливає на вартість підтримки цього модуля через рік?
- Управління ризиками: Якщо розрахована кількість помилок (B) > 0.5, які конкретні дії ви зробите як РМ (додаткові тести, Code Review, рефакторинг)?
- Порівняння: Чому об'єм програми (V) вважається кращою метрикою розміру, ніж просто кількість рядків коду (LOC)?
- Психологія: Як показник Effort (E) корелює з когнітивним навантаженням на програміста?

#### *Критерії оцінювання*

- Правильність ідентифікації токенів: 30%
- Точність розрахунків: 30%
- Якість менеджерського звіту: 40% (оцінюється вміння інтерпретувати цифри у бізнес-рішеннях).

#### *Приклад виконання завдання 1: Аналіз складності модуля CalculateShipping за методом Холстеда*

Вихідні дані (Варіант для прикладу): Модуль CalculateShipping.py

```
def calculate_shipping(weight, distance, is_vip):  
    base_rate = 50  
    if distance > 100:  
        multiplier = 1.5  
    else:  
        multiplier = 1.0  
    if weight > 20:
```

```

    extra_charge = (weight - 20) * 5
else:
    extra_charge = 0
total = (base_rate * multiplier) + extra_charge
if is_vip == True:
    total = total * 0.8 # 20% знижка для VIP
return total

```

### Крок 1. Лексичний аналіз (Ідентифікація токенів)

Розкладаємо код на базові складові. Для точності розрахунків важливо не пропустити жоден оператор (дію) чи операнд (дані).

Таблиця 1. Оператори (n1, N1)

| Оператор (унікальний) | Опис                  | Кількість (N1) |
|-----------------------|-----------------------|----------------|
| def                   | Оголошення функції    | 1              |
| calculate_shipping    | Ім'я функції          | 1              |
| () ,                  | Дужки та роздільники  | 5              |
| :                     | Початок блоку         | 4              |
| =                     | Присвоєння            | 6              |
| if, else              | Розгалуження          | 5              |
| >                     | Порівняння            | 2              |
| *, -, +               | Арифметика            | 5              |
| ==                    | Порівняння рівності   | 1              |
| return                | Повернення результату | 1              |
| Разом                 | n1=10 (унікальних)    | N1=31 (всього) |

Таблиця 2. Операнди (n2, N2)

| Операнд (унікальний) | Кількість (N2) | Операнд (унікальний) | Кількість (N2) |
|----------------------|----------------|----------------------|----------------|
| weight               | 2              | 20                   | 2              |
| distance             | 2              | extra_charge         | 3              |
| is_vip               | 2              | 5, 0                 | 2              |
| base_rate, 50        | 2              | total                | 4              |
| 100                  | 1              | True                 | 1              |
| multiplier           | 3              | 0.8                  | 1              |
| 1.5, 1.0             | 2              |                      |                |
| Разом                |                | n2=13 (унікальних)   | N2=27 (всього) |

### Крок 2. Розрахунок базових та фізичних метрик

#### 1. Словник програми (n):

$$n = n_1 + n_2 = 10 + 13 = \mathbf{23}$$

(Коментар РМ: Словник невеликий, що вказує на низький поріг входу для нового розробника).

#### 2. Довжина програми (N):

$$N = N_1 + N_2 = 31 + 27 = \mathbf{58}$$

#### 3. Об'єм програми (V):

$$V = N \cdot \log_2 n = 58 \cdot \log_2 23 \approx 58 \cdot 4.52 = \mathbf{262.16 \text{ біт}}$$

(Коментар РМ: Це об'єктивний розмір алгоритму в інформаційному просторі).

### Крок 3. Розрахунок інтелектуальних метрик (Складність та Зусилля)

#### 1. Складність ( $D$ ):

$$D = \frac{n_1}{2} \cdot \frac{N_2}{n_2} = \frac{10}{2} \cdot \frac{27}{13} = 5 \cdot 2.07 = \mathbf{10.35}$$

(Коментар РМ: Показник близько 10 є помірним. Код не є "спагеті", але вже має кілька точок потенційних помилок через інтенсивне використання операндів).

#### 2. Зусилля ( $E$ ):

$$E = V \cdot D = 262.16 \cdot 10.35 = \mathbf{2713.35} \text{ елементарних операцій}$$

### Крок 4. Прогноз часу та якості

#### 1. Час розробки ( $T$ ):

$$T = \frac{E}{18} = \frac{2713.35}{18} \approx 150.7 \text{ секунд} \approx \mathbf{2.5} \text{ хвилини}$$

(Коментар РМ: Це час суто на набір та базове обдумування. Для планування спринта я б заклав 1 годину з урахуванням написання Unit-тестів та документації).

#### 2. Ймовірність помилок ( $B$ ):

$$B = \frac{E^{2/3}}{3000} = \frac{(2713.35)^{0.66}}{3000} \approx \frac{194.5}{3000} \approx \mathbf{0.06}$$

(Коментар РМ: Значення  $B \ll 1$ . Це означає, що модуль дуже надійний. Шанс знайти критичний баг після написання — мінімальний).

### Крок 5. Managerial Report (Звіт менеджера)

Кому: Delivery Manager / Замовнику

Тема: Технічний аудит модуля CalculateShipping

За результатами аналізу методом Холстеда, модуль динамічного розрахунку доставки має низький рівень складності ( $D=10.35$ ) та високу прогнозовану надійність. Інтелектуальні зусилля на підтримку цього коду мінімальні завдяки оптимальному використанню змінних (операндів).

Рекомендація: Модуль готовий до інтеграції. Додаткове ретельне тестування не є критично необхідним через низький показник  $B(0.06)$ , достатньо стандартного покриття Unit-тестами основних бізнес-сценаріїв (VIP, вага  $> 20$  кг). Бюджет на підтримку цього модуля в майбутньому буде мінімальним.

Порада студентам: Зверніть увагу, як було перетворене сухе число 2713.35 (Зусилля) на висновок про те, що "бюджет на підтримку буде мінімальним". Саме за це РМ-ам і платять гроші — за інтерпретацію коду мовою бізнесу.

## Завдання №2. Управління технічним боргом та ризиками тестування через метрику Мак-Кейба

*Контекст (Кейс):* Ваша команда розробки підготувала критично важливий модуль для продукту над яким ви працювали (код модуля обрати самостійно на будь-якій мові розробки). Завтра – реліз. Розробники запевняють, що «все працює», але ви, як РМ, повинні прийняти рішення: релізити цей код чи відправити на рефакторинг, базуючись на цифрах.

### Крок 1. Побудова графу потоку управління (CFG)

Підібрати код модуля на власний розсуд, на будь-якій мові розробки. Що потрібно зробити:

- Побудувати (описати) Граф потоку управління (CFG) для цього модуля. Позначити всі вузли (блоки команд) та ребра (переходи).
- Виділити на графі предикатні вузли (точки прийняття рішень).

### Крок 2. Розрахунок метрик

1. Обчислити Цикломатичне число  $V(G)$  за одним із трьох способів:

- За кількістю регіонів графа.
- За формулою  $V(G)=e-n+2$ .
- За формулою  $V(G)=P+1$  (через предикатні вузли).

2. Визначити мінімально необхідну кількість тест-кейсів для повного покриття всіх логічних шляхів.

### Крок 3. Стратегія тестування (Менеджерський блок)

Складіть таблицю тест-кейсів (Basis Path Testing), які забезпечать 100% покриття. Для кожного тесту вкажіть вхідні дані (items\_count, total\_sum, etc.) та очікуваний результат.

| № Тесту | Вхідні дані (аргументи) | Шлях на графі (номери вузлів) | Очікуваний результат |
|---------|-------------------------|-------------------------------|----------------------|
| 1       | ...                     | ...                           | ...                  |

### Крок 4. Управлінське рішення

Дайте відповідь на наступні питання (обґрунтування має бути професійним):

- Оцінка ризику: Відповідно до стандартів NIST, до якої категорії складності відноситься цей модуль? Чи безпечний він для релізу без рефакторингу?
- Економіка: Уявіть, що написання та запуск одного тест-кейсу займає 30 хвилин. Скільки людино-годин потрібно вашому QA-відділу, щоб гарантувати відсутність логічних багів у цьому модулі?
- Рефакторинг: Якби  $V(G)$  було більше 20, які конкретні кроки ви б запропонували команді для зниження складності (наприклад, декомпозиція, патерни)?

*Критерії оцінювання:*

- Правильність побудови (опису) графа (коректне відображення вкладених if).
- Точність розрахунку  $V(G)$ .
- Повнота тестового покриття (чи всі шляхи враховані).
- Якість аргументації в управлінському рішенні.

*Приклад виконання завдання 2: Аналіз модуля CalculateOrderTotal.*

### Крок 1. Побудова графу потоку управління (CFG)

Вихідні дані (Варіант для прикладу): Модуль CalculateOrderTotal.py.

```
def calculate_order_total(items_count, total_sum, customer_type, is_heavy, promo_code):  
    final_price = total_sum  
    # 1. Базова логіка знижки  
    if customer_type == "LOYAL":
```

```

    if total_sum > 1000:
        final_price *= 0.9 # 10% знижка
    else:
        final_price *= 0.95 # 5% знижка
    elif customer_type == "NEW":
        if promo_code == "WELCOME2026":
            final_price -= 100
# 2. Логіка доставки
    if is_heavy:
        final_price += 500
        if total_sum < 5000:
            final_price += 200 # Доплата за габарити
    else:
        if total_sum > 2000:
            final_price += 0 # Безкоштовно
        else:
            final_price += 100 # Стандартна доставка
    return final_price

```

Розбиваємо код на логічні вузли. Кожна точка прийняття рішення (if/elif) – це розгалуження. Вузли графа:

- Node 1: Початок, ініціалізація final\_price.
- Node 2 (P1): if customer\_type == "LOYAL"
- Node 3 (P2): if total\_sum > 1000 (внутрішній для LOYAL)
- Node 4: final\_price \*= 0.9
- Node 5: final\_price \*= 0.95
- Node 6 (P3): elif customer\_type == "NEW"
- Node 7 (P4): if promo\_code == "WELCOME2026" (внутрішній для NEW)
- Node 8: final\_price -= 100
- Node 9: Точка злиття гілок дисконту (Junction).
- Node 10 (P5): if is\_heavy
- Node 11 (P6): if total\_sum < 5000 (внутрішній для Heavy)
- Node 12: final\_price += 200
- Node 13 (P7): if total\_sum > 2000 (внутрішній для Light)
- Node 14: final\_price += 0
- Node 15: final\_price += 100
- Node 16: Точка злиття гілок доставки.
- Node 17: return final\_price (Кінець).

## Крок 2. Розрахунок метрик

1) За предикатними вузлами (P): Предикати — це умови у коді:

1. customer\_type == "LOYAL"
2. total\_sum > 1000
3. customer\_type == "NEW"
4. promo\_code == "WELCOME2026"
5. is\_heavy
6. total\_sum < 5000
7. total\_sum > 2000

Разом: P=7, отже  $V(G) = 3+1 = 7+1=8$ .

2) За кількістю регіонів: Якщо намалювати цей граф на папері, він утворить 7 замкнених циклів (областей) + 1 зовнішня область:  $V(G)=8$ . Висновок: Для повного покриття логіки (Branch Coverage) нам потрібно мінімум 8 тест-кейсів.

### Крок 3. Стратегія тестування (Basis Path)

QA-команді необхідно перевірити ці 8 критичних шляхів:

| № Тесту | Тип клієнта | Сума | Heavy? | Промокод    | Очікуваний шлях (вузли)    |
|---------|-------------|------|--------|-------------|----------------------------|
| 1       | LOYAL       | 1500 | Yes    | -           | 1-2-3-4-9-10-11-12-16-17   |
| 2       | LOYAL       | 500  | Yes    | -           | 1-2-3-5-9-10-11-12-16-17   |
| 3       | LOYAL       | 6000 | Yes    | -           | 1-2-3-4-9-10-11-16-17      |
| 4       | NEW         | 500  | No     | WELCOME2026 | 1-2-6-7-8-9-10-13-15-16-17 |
| 5       | NEW         | 500  | No     | NO PROMO    | 1-2-6-7-9-10-13-15-16-17   |
| 6       | OTHER       | 3000 | No     | -           | 1-2-6-9-10-13-14-16-17     |
| 7       | OTHER       | 1000 | No     | -           | 1-2-6-9-10-13-15-16-17     |
| 8       | LOYAL       | 1500 | No     | -           | 1-2-3-4-9-10-13-14-16-17   |

### Крок 4. Управлінське рішення

1) Оцінка ризику: Згідно з NIST,  $V(G)=8$  потрапляє в категорію «Проста програма» (1-10).

- Вердикт: Модуль безпечний для релізу за умови проходження всіх 8 тестів. Код не є «спагеті», але він на межі. Якщо маркетологи захочуть додати ще 3-4 умови, складність перетне поріг 10, і нам знадобиться рефакторинг.

2) Економіка (Ресурсне планування):

- Кількість тестів: 8.
- Час на тест: 0.5 години.
- Загальний час:  $8 \cdot 0.5 = 4$  людино-години.
- PM Insight: Якщо розробник каже, що протестує все за 15 хвилин — він бреше. Мені потрібно закласти мінімум пів робочого дня QA-інженера в план спринту на цей модуль.

3) Рефакторинг (Майбутній розвиток):

Зараз код написаний одним «простирадлом». Щоб складність не зростала, я запропоную розбити його на два незалежні модулі (декомпозиція):

- `get_discount_rate(customer_type, total_sum, promo_code)` — складність знизиться до 5.
- `get_shipping_fee(is_heavy, total_sum)` — складність знизиться до 4.
- Результат: Сумарна складність системи залишиться такою ж, але кожен окремий блок буде набагато легше читати та підтримувати (SRP - Single Responsibility Principle).

*Порада викладачу:* При перевірці звертайте увагу на те, чи не забули студенти шлях «OTHER» (коли клієнт не LOYAL і не NEW). Це найпоширеніша помилка, яка призводить до багів у продакшені.

### Завдання 3. Аудит інформаційної складності ПЗ за метрикою Чепіна

*Контекст (Кейс):* Ваша команда отримала у спадок (Legacy) деякий розроблений модуль продукту над яким ви працюєте. Замовник скаржиться, що внесення будь-яких змін у цей код займає забагато часу і супроводжується великою кількістю багів. Вам, як РМ-у, потрібно провести аудит цього модуля.

### Крок 1. Аналіз вихідного коду

Невеликий технічний опис програмного модуля.

## Крок 2. Класифікація змінних та розрахунок Q

Заповніть аналітичну таблицю, розподіливши всі змінні функції за класифікацією Неда Чепіна.

| Змінна        | Тип (P, M, C, T) | Обґрунтування ролі | Вага (1, 2, 3, 0.5) |
|---------------|------------------|--------------------|---------------------|
| weight        |                  |                    |                     |
| distance      |                  |                    |                     |
| shipping_type |                  |                    |                     |
| ...           |                  |                    |                     |

Розрахуйте інтегральний показник за формулою:

$$Q = 1 \cdot P + 2 \cdot M + 3 \cdot C + 0.5 \cdot T$$

## Крок 3. Управлінський висновок

Дайте відповідь на наступні питання:

- Аналіз ризиків: Яка категорія змінних (P, M, C чи T) зробила найбільший внесок у фінальне значення Q? Як це впливає на складність тестування даного модуля?
- Прогноз підтримки: Як зміниться показник Q, якщо замовник вимагатиме додати ще одну умову — «Безкоштовна доставка для замовлень понад 5000 грн»? Яку нову змінну ви введете і до якого типу вона належатиме?
- Оптимізація: Запропонуйте архітектурне рішення для зменшення складності Q (наприклад, декомпозиція). Як розподіл цієї функції на дві менші змінить когнітивне навантаження на розробника?

*Критерії оцінювання:*

- Правильність класифікації: Чи правильно визначено тип кожної змінної (особливо C та M).
- Точність розрахунків: Коректність фінального числа Q.
- Якість аргументації: Здатність пояснити технічні метрики мовою бізнесу (ризики, час, гроші).

*Приклад виконання завдання 3:* Аудит модуля CalculateShipping

### Крок 1. Аналіз вихідного коду

Код виконує розрахунок вартості доставки, базуючись на фізичних параметрах (вага, відстань) та бізнес-правилах (типи доставки, статуси клієнтів). Бачимо багато розгалужень (if/elif), що свідчить про високу логічну навантаженість. Нижче наведено код функції розрахунку доставки.

```
def calculate_shipping(weight, distance, shipping_type, client_status, is_fragile, warehouse_id):  
    # Тимчасовий ідентифікатор для внутрішнього логування (не впливає на логіку)  
    trace_id = f"W{warehouse_id}-D{distance}"  
    base_rate = 50  
    final_cost = 0  
    # Визначення базового тарифу за типом доставки  
    if shipping_type == "EXPRESS":  
        multiplier = 2.5  
    elif shipping_type == "STANDARD":  
        multiplier = 1.0  
    else:  
        multiplier = 0.8 # Economy  
    # Націнка за вагу та відстань  
    weight_fee = weight * 1.5  
    distance_fee = distance * 0.1
```

```

# Логіка для крихких товарів
if is_fragile:
    fragile_insurance = 100
else:
    fragile_insurance = 0
# Врахування статусу клієнта для знижки
if client_status == "GOLD":
    discount = 0.20
elif client_status == "SILVER":
    discount = 0.10
else:
    discount = 0
final_cost = (base_rate * multiplier) + weight_fee + distance_fee + fragile_insurance
final_cost = final_cost * (1 - discount)
return final_cost

```

## Крок 2. Класифікація змінних та розрахунок Q

Проаналізуємо кожну змінну, що з'являється у коді:

| Змінна            | Категорія    | Обґрунтування ролі                                                      | Вага |
|-------------------|--------------|-------------------------------------------------------------------------|------|
| weight            | P (Input)    | Використовується суто як вхідний параметр для розрахунку weight fee.    | 1    |
| distance          | P (Input)    | Вхідний параметр для математичного розрахунку distance fee.             | 1    |
| shipping_type     | C (Control)  | Керує вибором гілки if/elif для визначення multiplier.                  | 3    |
| client_status     | C (Control)  | Керує логікою визначення розміру знижки (discount).                     | 3    |
| is_fragile        | C (Control)  | Визначає, чи буде нарахована страховка fragile insurance.               | 3    |
| warehouse_id      | T (Trash)    | Входить до складу trace_id, але не впливає на фінальну ціну замовлення. | 0.5  |
| trace_id          | T (Trash)    | Служить лише для логування, результат розрахунку від неї не залежить.   | 0.5  |
| multiplier        | M (Modified) | Створюється та змінюється всередині модуля залежно від умов.            | 2    |
| base_rate         | M (Modified) | Хоча вона ініціалізована як 50, вона є внутрішнім станом розрахунку.    | 2    |
| weight fee        | M (Modified) | Проміжна розрахункова змінна.                                           | 2    |
| distance fee      | M (Modified) | Проміжна розрахункова змінна.                                           | 2    |
| fragile insurance | M (Modified) | Змінна, що змінює стан залежно від умов.                                | 2    |
| discount          | M (Modified) | Змінна, що модифікується в процесі роботи алгоритму.                    | 2    |
| final_cost        | M (Modified) | Результуюча змінна, що зазнає декількох трансформацій.                  | 2    |

Розрахунок показника Q:

1. **P** (2 змінні):  $2 \times 1 = 2$

2. **M** (7 змінних):  $7 \times 2 = 14$

3. **C** (3 змінні):  $3 \times 3 = 9$

4. **T** (2 змінні):  $2 \times 0.5 = 1$

**Разом: Q = 2 + 14 + 9 + 1 = 26**

### Крок 3. Управлінський висновок (PM Report)

#### 1. Аналіз ризиків

Найбільший внесок у Q зробили змінні типу M (Modified) — 14 балів, та C (Control) — 9 балів. Велика кількість модифікованих змінних свідчить про те, що функція перевантажена проміжними розрахунками. 3 керуючі змінні (C) означають, що у нас є мінімум  $3 \times 3 = 9$  базових комбінацій для тестування. Ризик помилки високий, бо зміна однієї керуючої змінної (наприклад, `shipping_type`) каскадом впливає на `multiplier`, а потім на `final_cost`.

#### 2. Прогноз підтримки

Якщо замовник додасть умову «Безкоштовна доставка понад 5000 грн», нам доведеться ввести нову керуючу змінну (наприклад, `order_value`) або використати існуючу в новій логічній гілці. Це додасть мінімум 3 бали до показника Q (категорія C). Складність зросте до 29, що наближає модуль до межі "важкої підтримки". Кожне таке "просте" прохання замовника робить код все більш заплутаним.

#### 3. Оптимізація

Я рекомендую провести декомпозицію. Модуль слід розбити на три підфункції:

- `get_multiplier(shipping_type)`
- `get_insurance(is_fragile)`
- `get_discount(client_status)`

Це дозволить винести всі 9 балів складності керуючих змінних (C) в окремі дрібні модулі. Основна функція `calculate_shipping` тоді буде містити лише змінні типу P та M, її особистий Q впаде до ~10-12. Це зробить код прозорим, а тестування — ізольованим і дешевим.